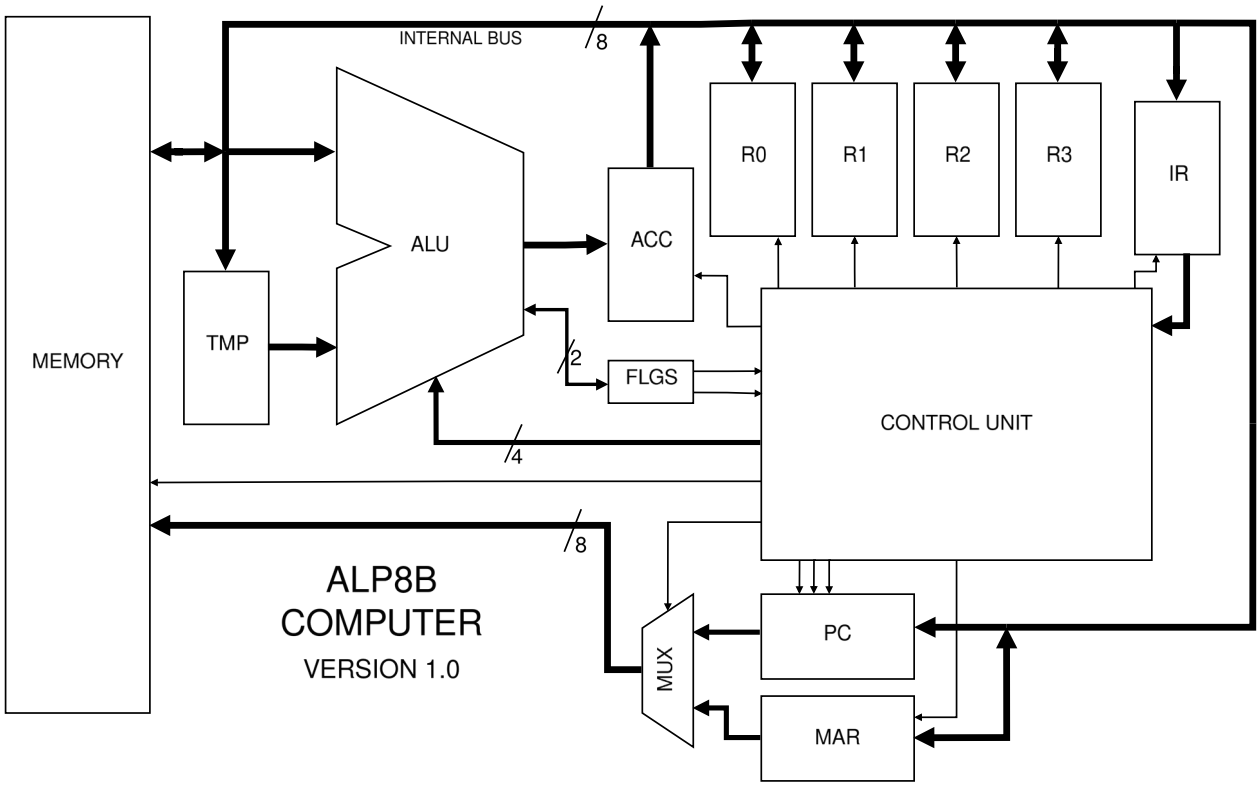
# ALP8B Computer Project

**By Alper Alpcan**

Revision 1.0

2026/08/02



# Table of Contents

# 1 Project Scope & Summary

The ALP8B (ALPer’s 8-Bit) is a custom 8-bit computer operating on a self-designed Instruction Set Architecture (ISA) and a custom simulated hardware implementation. It operates within a pure 256-byte Von Neumann memory space, where the "display" to the user is a direct window into the memory itself, formatted as a grid.

The primary goal of this project was to serve as a stepping stone into the basics of computer architecture and design and thus it is not intended to be anything more than an educational system. Beyond the learning value, writing software for such a constrained machine presented a unique and interesting puzzle-solving challenge. It was an incredibly rewarding experience to write small demoscene programs for a system I designed from the ground up.

Below is a quick summary of the technical specifications that are explained in detail throughout this document:

- **Hardware:** 4 general-purpose registers, 1 special flags register, an 8-bit ALU, and a custom hybrid microcoded Control Unit.
- **Software:** A two-pass Python assembler and microcode compiler, along with a variety of demoscene programs to demonstrate the system's capabilities.

This document serves as both a technical reference and a source for the reasoning behind the design decisions made during development.

**Notice of AI usage:**

No AI was used in the direct implementation of systems and circuitry as AI (obviously) cannot interact with the simulation software. However, I did use AI for research, asking questions, some debugging and also just brainstorming ideas and implementation methods. I also occasionally used AI to clean up the wording and grammar in this document.

# 2 ISA Overview & Reference Table

The architecture of the ALP8B is a custom 8-bit, Load/Store RISC-inspired Instruction Set Architecture. It utilises a (mostly) single byte instruction width to ensure code density within an 8-bit Von Neumann instruction/data memory space.

## 2.1 Core Specifications

- Word size: 8-bit
- Instruction length: Majority 1-word, 2-word special
- Register file: Four 8-bit General Purpose Registers R0, R1, R2, R3. These are represented by fields RA & RB in instructions (00 for R0, 01 for R1, 10 for R2, and 11 for R3).
- Condition flags: Two status flags, Carry (C) and Zero (Z).

## 2.2 Instruction Encoding

The ISA is designed with a 2-bit opcode in mind, with a 2-bit opcode extension for additional instruction density. The four major regions are:

- 00: Variable, used for “housekeeping” instructions and additional ALU instructions.
- 01: Branching instructions.
- 10: ALU instructions.
- 11: Memory related instructions.

Instructions are classified into one of three formats:

1. Format A (No Registers):

```
[Opcode (2b) | Ext 1 (2b) | Ext 2 (2b) | Ext 3 (2b)]
```

2. Format B (Single Register):

```
[Opcode (2b) | Ext 1 (2b) | Ext 2 (2b) | RB    (2b)]
```

3. Format C (Dual Register):

```
[Opcode (2b) | Ext 1 (2b) | RA    (2b) | RB    (2b)]
```

## 2.3 Reference Tables

The following tables are a full outline of the ISA. For the Flags column, ‘-’ means unaffected, ‘*’ means updated, and a value of 0/1 means unconditional set. Cycle count is indexed from 0, inclusive of the mandatory 2 cycles required for fetching, and the step to trigger the reset itself.

### 2.3.1 Format A Instructions

| Encoding [7:0] | Mnemonic | Operands | Operation | Description | Flags (Z, C) | Size (Words) | Cycles |
|---|---|---|---|---|---|---|---|
| `0000 0000` | `HLT` | None | Halt | Halts the core clock. | - , - | 1 | 3 |
| `0000 0001` | `NOP` | None | Nop | No operation. | -, - | 1 | 3 |
| `0000 0010` | `CLC` | None | C ← 0 | Clear Carry status flag. | -, 0 | 1 | 3 |
| `0000 0011` | `SEC` | None | C ← 1 | Set Carry status flag. | -, 1 | 1 | 3 |
| `W1:`<br>`0100 0000`<br>`W2:`<br>`AAAA AAAA` | `JMP` | Imm8 | PC ← Imm8 | Unconditional jump to address in W2. | -, - | 2 | 3 |
| `W1:`<br>`0100 0001`<br>`W2:`<br>`AAAA AAAA` | `JEQ` | Imm8 | If (Z = 1, C = X):<br>PC ← Imm8 | Jumps to address in W2 if Equal condition is met. (Note: Use CMP prior to branching to update flags). | -, - | 2 | 3 |
| `W1:`<br>`0100 0010`<br>`W2:`<br>`AAAA AAAA` | `JLT` | Imm8 | If (Z = 0, C = 0):<br>PC ← Imm8 | Jumps to address in W2 if Less Than condition is met. (Note: Use CMP prior to branching to update flags). | -, - | 2 | 3 |
| `W1:`<br>`0100 0011`<br>`W2:`<br>`AAAA AAAA` | `JGT` | Imm8 | If (Z = 0, C = 1):<br>PC ← Imm8 | Jumps to address in W2 if Greater Than condition is met. (Note: Use CMP prior to branching to update flags). | -, - | 2 | 3 |

### 2.3.2 Format B Instructions

| Encoding [7:0] | Mnemonic | Operands | Operation | Description | Flags (Z, C) | Size (Words) | Cycles |
|---|---|---|---|---|---|---|---|
| `0000 01RB` | `SHL` | RB | RB ← RB << 1 | Logical left shift by 1. | *, * | 1 | 4 |
| `0000 10RB` | `SHR` | RB | RB ← RB >> 1 | Logical right shift by 1. | *, * | 1 | 4 |
| `0000 11RB` | `NOT` | RB | RB ← ¬RB | Negation of RB. | *, * | 1 | 4 |
| `0001 00RB` | `INC` | RB | RB ← RB + 1 | Increment by 1. | -, - | 1 | 4 |
| `0001 01RB` | `DEC` | RB | RB ← RB - 1 | Decrement by 1. | -, - | 1 | 4 |

### 2.3.3 Format C Instructions

| Encoding [7:0] | Mnemonic | Operands | Operation | Description | Flags (Z, C) | Size (Words) | Cycles |
|---|---|---|---|---|---|---|---|
| `0010 RARB` | `CMP` | RA, RB | Flags ← Status(RA - RB) | Compare RA with RB, Performs RA - RB internally with no save. | *, * | 1 | 4 |
| `0011 RARB` | `SUB` | RA, RB | RA ← RA - RB | Subtraction without borrow. | *, * | 1 | 5 |
| `1000 RARB` | `ADC` | RA, RB | RA ← RA + RB + C | Addition with carry. | *, * | 1 | 5 |
| `1001 RARB` | `ORR` | RA, RB | RA ← RA ∨ RB | Logical OR. | *, * | 1 | 5 |
| `1010 RARB` | `AND` | RA, RB | RA ← RA ∧ RB | Logical AND. | *, * | 1 | 5 |
| `1011 RARB` | `XOR` | RA, RB | RA ← RA ⊕ RB | Logical XOR. | *, * | 1 | 5 |
| `1100 RARB` | `LDM` | RA, RB | RA ← M[RB] | Memory indirect load | -, - | 1 | 4 |
| `1101 RARB` | `STM` | RA, RB | M[RB] ← RA | Memory indirect store | -, - | 1 | 4 |
| `1110 RARB` | `MOV` | RA, RB | RA ← RB | Copy value of RB into RA | -, - | 1 | 3 |
| `W1:`<br>`1111 RAXX`<br>`W2:`<br>`VVVV VVVV` | `LDI` | RA, Imm8 | RA ← Imm8 | Set RA to the value stored in the next word. | -, - | 2 | 4 |

# 3 ISA Design Decisions

## 3.1 Architectural Scope

The ALP8B was designed with a strict 256 byte Von Neumann memory size. This is due to this project mainly being an education introduction into basic computer architecture and design. If the ALP8B were to have an 16 bit address space of memory, many of the instructions that need to hold an address value (branching & immediate instructions), would be at minimum 3 bytes long. This would exponentially increase hardware complexity and the difficulty of the project. The ISA was not designed with potential expansion for the future in mind due to the ‘tech-demo’ nature of this project.

## 3.2 ALU Optimisation & Quirks

To simplify hardware implementation of the ISA, the `SUB`, `CMP` and `ADC` instruction all use the same 8-bit adder circuitry. `SUB/CMP` were implemented via two’s complement inversion `A + (¬B) + 1`, with the ‘1’ hardwired through the carry in bit to the adder, hence why the SUB instruction doesn’t consume the carry and isn’t ‘`SBC`’. Additionally, the `CMP` instruction shares identical wiring as the `SUB` instruction – the difference in functionality is within the microcode sequence. This is once again a choice to reduce hardware complexity at the cost of a reduced set of functionality.

Due to the limiting size of an 8-bit instruction, ALU operations are spread across both the 00 and 10 block. Although this slightly ruins ISA orthogonality and adds some more logic to the instruction decoder, it was considered a reasonable trade off.

## 3.3 Branching

As seen in the reference table, all branching instructions are two bytes wide, with the second byte storing the immediate memory address. As mentioned previously, this is simplified by the 256 byte memory space (can be addressed fully by a single byte). A majority of the inspiration for the branching instructions came from the 6502 architecture.

The decision to have a dedicated `JGT`, rather than following the MIPS/RISC-V route of keeping it a pseudo-instruction was made due to the following reasons:

- Increased simplicity of the software assembler
- Greater instruction density, as having a synthesised greater than branch would need to compile into two instructions.
- Limitations due to only having two flags to identify branching status.

As branching instructions themselves do not complete a comparison, it is recommended that an explicit `CMP` is used before each branch, as to prevent accidental execution of a jump due to stale flags.

## 3.4 Memory Indirect Addressing

The decision to use memory indirect addressing was made to limit the amount of costly 2-byte wide instructions, as an immediate addressing instruction would require two bytes width. A 2-byte width would be expensive as `LTM` / `STM` are commonly used instructions. Once again, the 256 memory space is easily addressed by the value inside a single register, hence allowing for simpler hardware.

This ties into the decision to make the `INC` / `DEC` instructions not affect the condition flags, as these can thus be used to move a “pointer” register around easily to access sequential memory locations without tampering with the flag state(s) used for branching.

## 3.5 Other Decisions

1. `HLT` is assigned `0000 0000` as a preventative measure to ensure that if execution goes out of bounds, the computer safely stops.
2. Explicit `NOP` was added rather than a pseudo implementation via `MOV RA RA` or similar due to unallocated opcode space within the Format A decoding map in the 00 block.
3. The ISA features no stack and no stack related instructions as they were deemed too expensive for a 256 byte memory space. Same reasoning applies for interrupts.

# 4 Hardware Implementation

***Note:*** *The ALP8B was designed in logic simulation software only (Digital by Hneemann). As this was a learning project, optimising for 74x chips and other real world limitations and imperfections was not strictly considered.*

## 4.1 System Microarchitecture & Block Diagram

As seen in Figure 4.1, the ALP8B is a single bus, 9 register computer, with a dedicated 2-bit conditional flag register (FLGS). The four general purpose registers from the ISA can be seen labelled R0...R3, other registers include: Program Counter (PC) Instruction Register (IR), Memory Address Register (MAR), ALU temporary input (TMP), and ALU accumulator (ACC). The Control Unit (CU) represents the region that decodes instructions and controls the master clock.

### 4.1.1 Summary of Execution Cycle

The standard fetch-decode-execute path on the ALP8B is as follows.

1. Fetch: The CU sets the MUX to take the address input from the PC, enables output from the RAM, and enables write to IR. Then increments the PC using its internal half adder
2. Decode/Execute: A microcode ROM is used to enable or disable required control signals. The microcode sequence is addressed by the time state and the top 4 bits of the instruction.

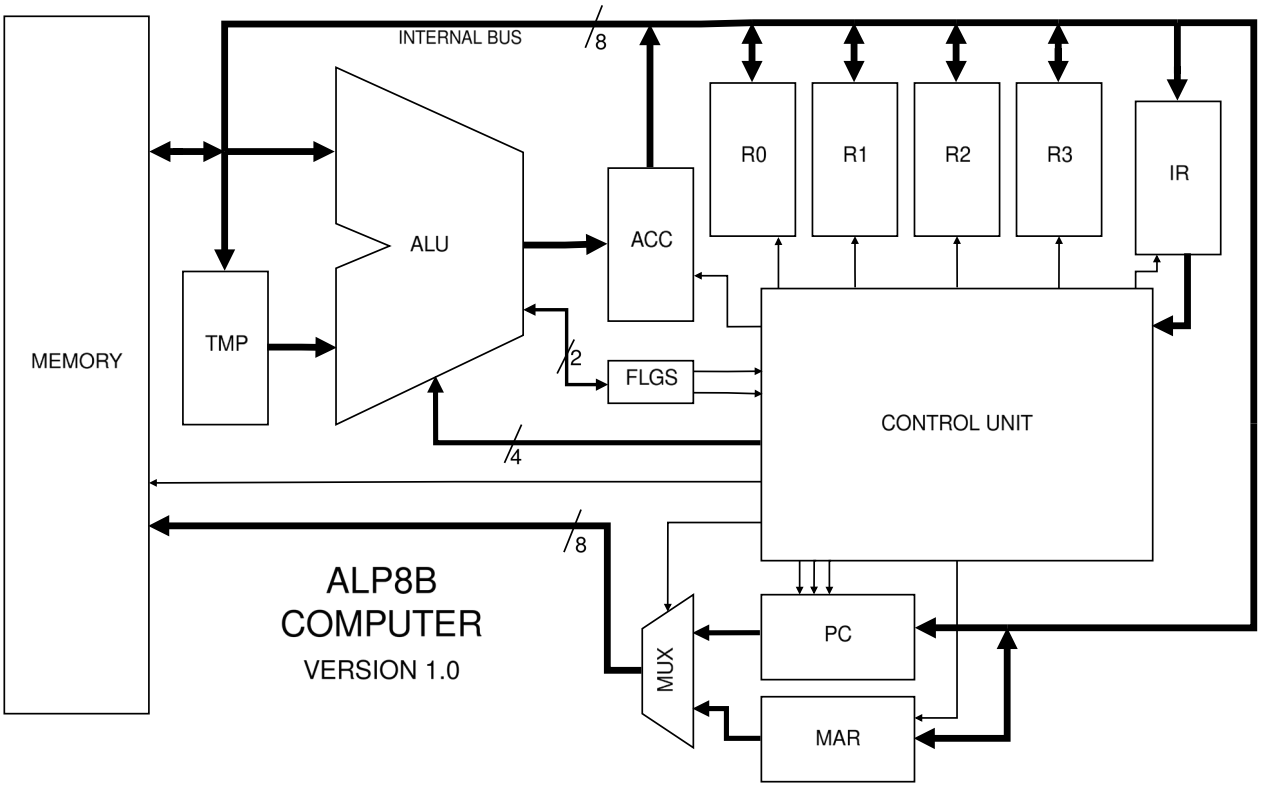



*Figure 4.1: Block diagram of the ALP8B*

## 4.2 Control Unit

### 4.2.1 Sequencer

The ALP8B is designed around a 6 stage instruction cycle. Time steps T0-T5 are generated through a counter. Early resetting of the counter is done synchronously, and is triggered by one of three signals; a microcode sequence early reset, hitting T6 (used as a loopback reset), or a `NOP` instruction. Although instructions have the ability to use all steps from T2-T5, all instruction sequences currently only need up to T3 before calling a reset on the following step.

T0 and T1 are hardwired to handle the fetch cycle, with T0 handling instruction retrieval and T1 sending a pulse to increment the PC. T0 and T1 are wired to a “FETCH” signal that disables all microcode ROM output until T2 onwards.

T2-T5 are encoded into a 2-bit number and are fed into the address lines of the microcode ROM to index the sequence steps of a given instruction.

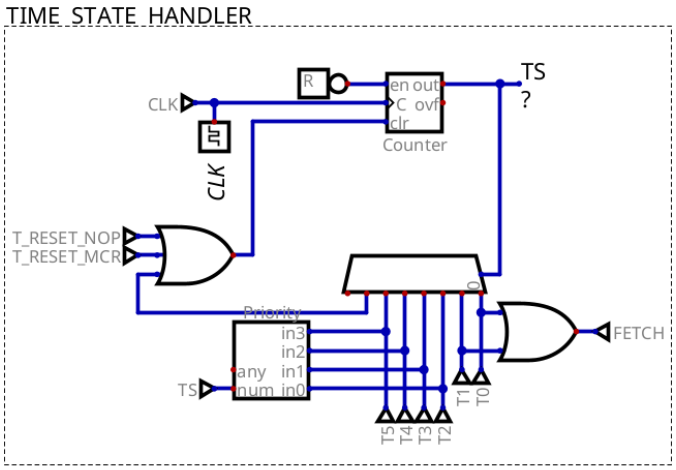



*Figure 4.2: Time state sequencer implementation in Digital*

### 4.2.2 Microcode ROM

**Why Microcode?**

A hybrid microcode architecture for the Control Unit was chosen for a couple of reasons:

- Ease of bug fixing and simple reprogramming when control paths require changes
- Reducing visual logic gate clutter
- Desire to learn how microcoded systems work on a fundamental level

The microcode ROM is a 6-bit address and 8-bit data memory, resulting in a total of sixteen, four step microcode sequences.

Due to the 8-bit output, the control bits are decoded outside of the ROM, allowing for more control signals to be packed into 8 bits. The following table explains what each of the bits represent:

| Bit 7 | Bit 6 | Bits 5-3 | Bit 2 | Bits 1-0 |
|---|---|---|---|---|
| VALID | T_RESET | DEST | ADDRSEL / enFLGS | SRC |

**VALID:** All microcode sequence bytes must have this bit HI. Microcode decoders are locked to only enable when T_RESET is LO and this bit is HI in order to prevent invalid control signals being sent during a sequence reset.

**T_RESET:** Efficiency bit, HI to signal the early exit of a sequence.

**DEST:** Destination location (write enable control), 3-bit encoded signal, control signal mapping below:

| | |
|---|---|
| 000 | RA – IR bits used to determine which register |
| 001 | RB – IR bits used to determine which register |
| 010 | ACC – ALU accumulator only. Note: See ADDRESSEE / enFLGS below |
| 011 | FLGS – ALU flags only |
| 100 | TMP – ALU temporary |
| 101 | PC – increment / enable jump |
| 110 | MAR – memory address register |
| 111 | RAM – memory |

**ADDRSEL / enFLGS:** This is a dual purpose bit, when HI, it sets the MUX in Figure 4.1 to use the MAR for indexing the memory. Simultaneously, when HI at the same time as DEST 010 (ACC), it allows both the FLGS and ACC registers to be write enabled at the same time, which is otherwise impossible with the decoder. This is used for instructions that need to save both their output and condition flag updates. Due to the instruction set design, these two purposes are mutually exclusive.

**SRC:** Source location (output enable control), 2 bit encoded signal, mapping below:

| | |
|---|---|
| 00 | RA – IR bits used to determine which register |
| 01 | RB – IR bits used to determine which register |
| 10 | ACC – ALU accumulator |
| 11 | RAM – memory |

### 4.2.2.1 Microcode enabling

The microcode ROM decoders are only active if: the instruction is not in the “Housekeeping” op region (00 00 00 XY), the time state is not T0/T1, and there is currently no T_RESET active. This is the “ROM ACTIVE MASK” section below.

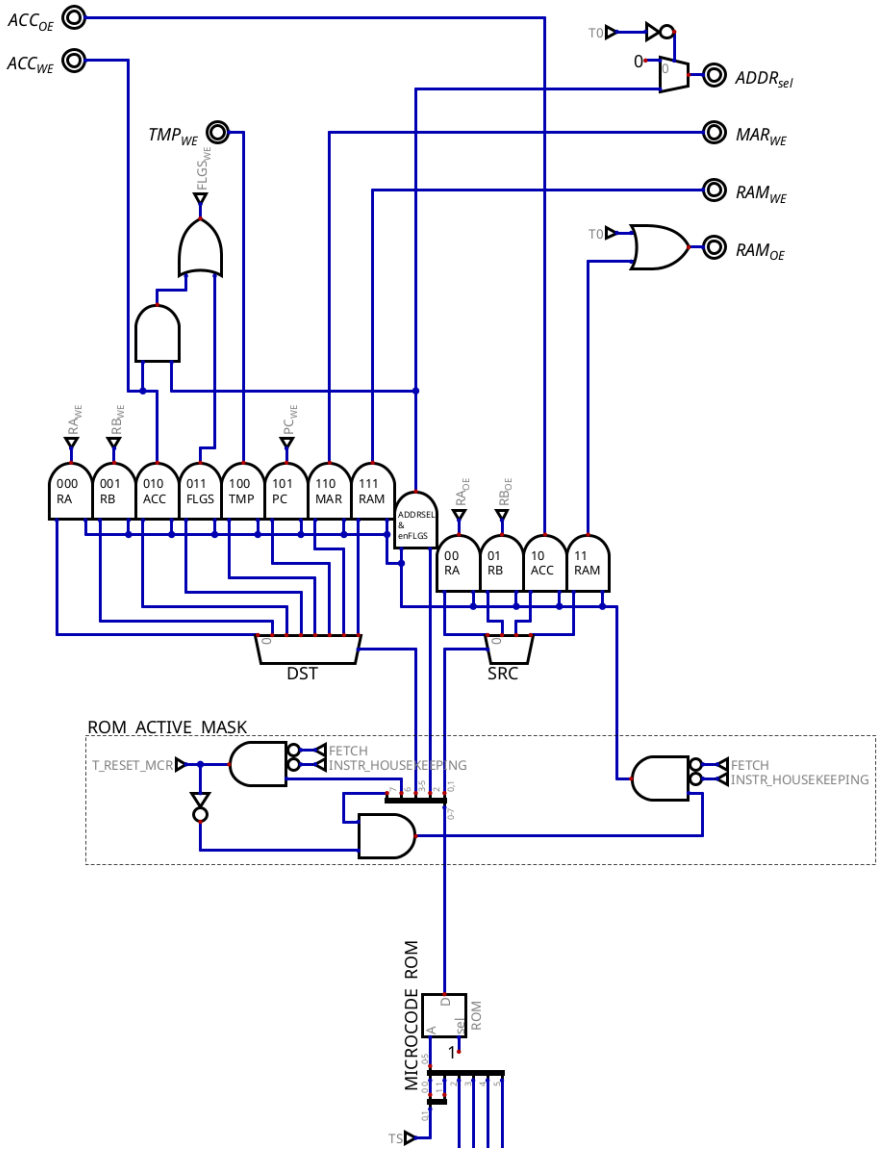



*Figure 4.3: Instruction control signal decoding in Digital via the microcode ROM*

### 4.2.2.2 Future Considerations

In hindsight, the decoder activation gates could’ve been avoided entirely if the design of the microcode data word was done more carefully. Specifically, a much simpler system could’ve been made if empty bits in the microcode word did not map to valid control signals (i.e., zeros do not decode to a control signal). See 4.4.

### 4.2.3 Program Counter

The program counter was designed with an internal half adder circuit, which allows for incrementing without using the dedicated ALU, saving clock cycles.

The D flip flops used in the program counter are async reset master slave D flip flops. A pulse is sent to RST to clear the program counter from noise values and set to 0x00 on start.

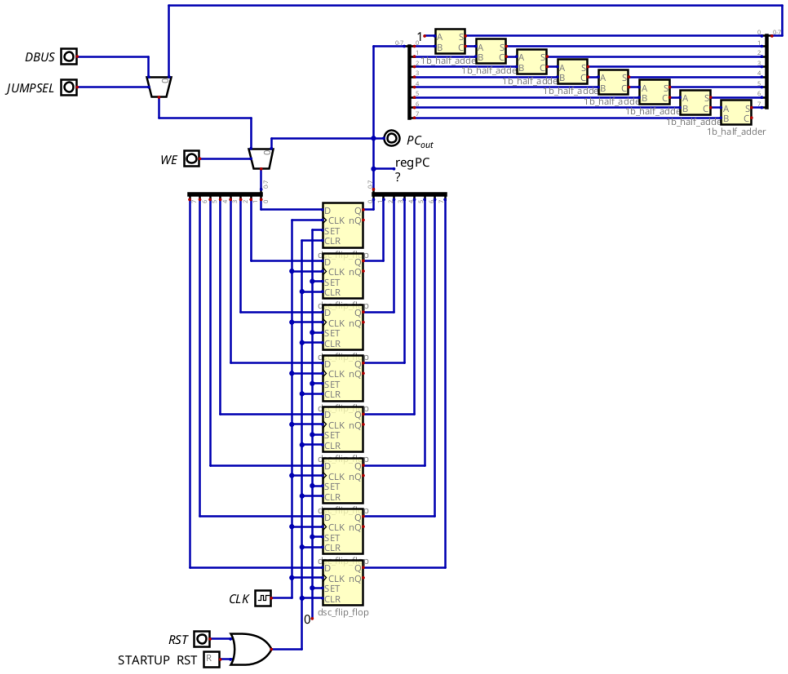



*Figure 4.4: Program counter implementation in Digital*

Two control signals are used to control incrementation / jumping. A normal Write Enable HI will simply feed in the incremented value, however, if WE is HI and JUMPSEL is HI, the D flip flops will read in the value from the data bus, which contains the jump address. This is implemented in the Control Unit as seen below. Note as of writing, resetting the PC has not been implemented.

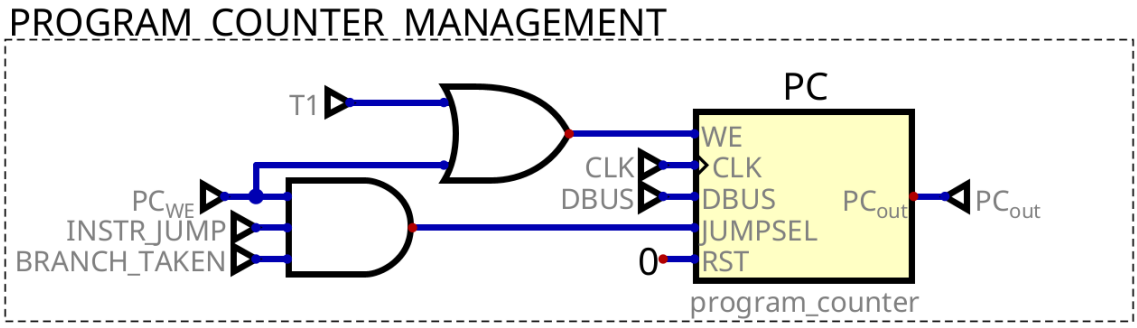



*Figure 4.5: PC implementation in the Control Unit*

### 4.2.3.1 Branching

As seen above in Figure 4.5, the JUMPSEL pin is controlled by three signals, two that are rather trivial, INSTR_JUMP means we are in the opcode region for branching instructions, and PC_WE means that a write to the PC is to occur.

The third BRANCH_TAKEN signal, however, is utilised to determine whether a jump in the program counter is actually supposed to take place – this is used as all JUMP instructions use the same microcode sequence, therefore the logic to determine whether the carry flags are to cause a jump lives outside of the microcode entirely. Bits 6 and 7 (to determine which branch instruction), ZERO, and CARRY are used in a hardwired manner to determine if the branch is to be taken or not.The logic for the BRANCH_TAKEN signal was synthesised via Digital's truth table solver.

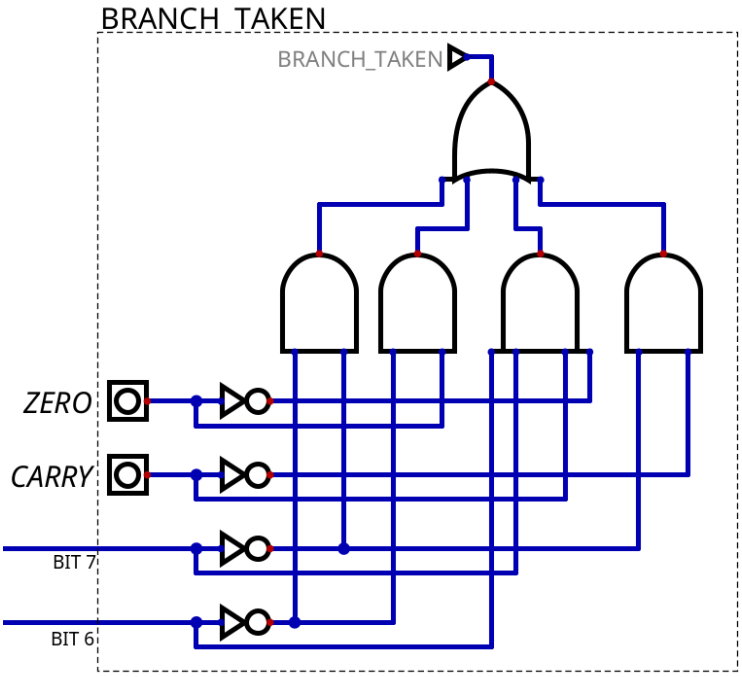



*Figure 4.6: Hardwired branch enabler implementation*

### 4.2.4 Hardwired “Housekeeping” Instructions

As seen in the ISA, there are four instructions without operands. These instructions, dubbed the “Housekeeping” instructions, are hardwired and thus are not controlled via the microcode ROM. The simple implementation is that when the top 6 bits of the instruction are 0, then disable microcode (4.2.2.1 ) and instead use the following implementation shown below.

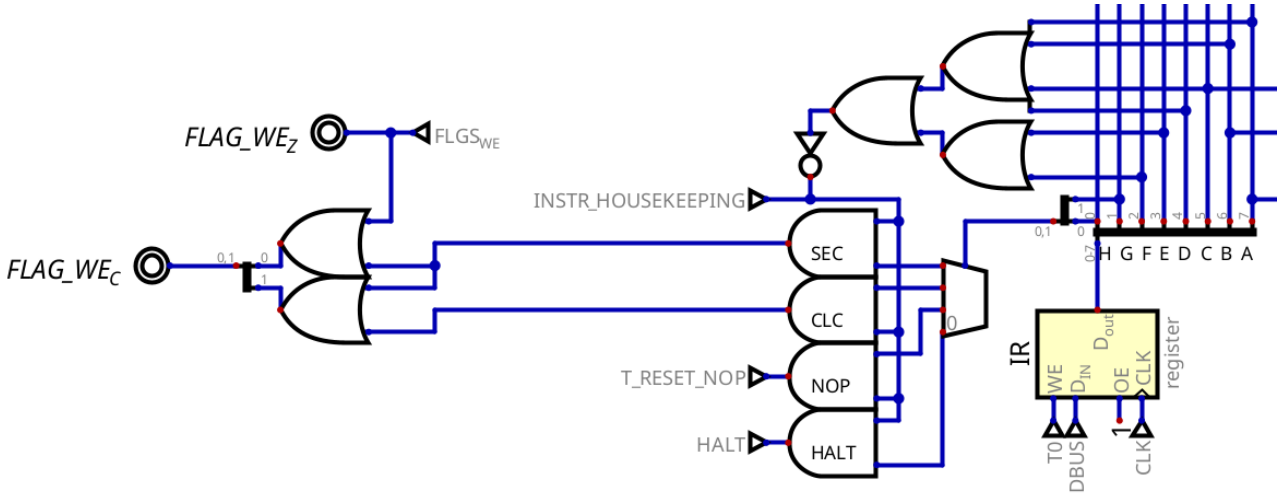



*Figure 4.7: Housekeeping instructions hardwiring*

Refer to Section 4.4.1 for understanding on how the 2-bit code for setting the flag register is handled.

## 4.3 Arithmetic Logic Unit (ALU)

The ALP8B has 11 instructions that are routed through the ALU for execution, 5 instructions that are single register (Format B), and 6 instructions that are dual register (Format C). Thus, a 4-bit ALU opcode is used to represent these instructions. ALU opcodes were, naively, added out of order as the ISA was expanded multiple times during development, this has led to a potentially suboptimal mapping of instruction opcode to ALU opcode. The mappings can be seen below. Note the use of 'RR' to represent bits given to register selection (and thus ignored in mapping logic).

| Single register operations (Format B) | | |
|---|---|---|
| Mnemonic | Opcode (8b) | ALU-op (4b) |
| SHL | 0000 01RR | 0000 |
| SHR | 0000 10RR | 0001 |
| NOT | 0000 11RR | 0010 |
| INC | 0001 00RR | 0011 |
| DEC | 0001 01RR | 0111 |

| Dual register operations (Format C) | | |
|---|---|---|
| Mnemonic | Opcode (8b) | ALU-op (4b) |
| ADC | 1000 RRRR | 1000 |
| ORR | 1001 RRRR | 1001 |
| AND | 1010 RRRR | 1010 |
| XOR | 1011 RRRR | 1011 |
| CMP | 0010 RRRR | 0100 |
| SUB | 0011 RRRR | 0101 |

*Table 1: Instruction to ALU opcode mappings*

As a split between ALU instructions between two opcode regions was required in the ISA to fit all instructions, the mapping stage is done in two stages, one for each opcode region. This means that regions 00 and 10 are mapped simultaneously to their correct op code instruction, and only enabled to travel to the ALU when the instruction type is either in region 00 or 10. During instructions that are in other regions, the ALU receives code 0000, although it does not matter as no register will be listening.

Note: As per 4.2, the microcode is not active during housekeeping instructions, so they cannot conflict with ALU operations or ALU-op mapping

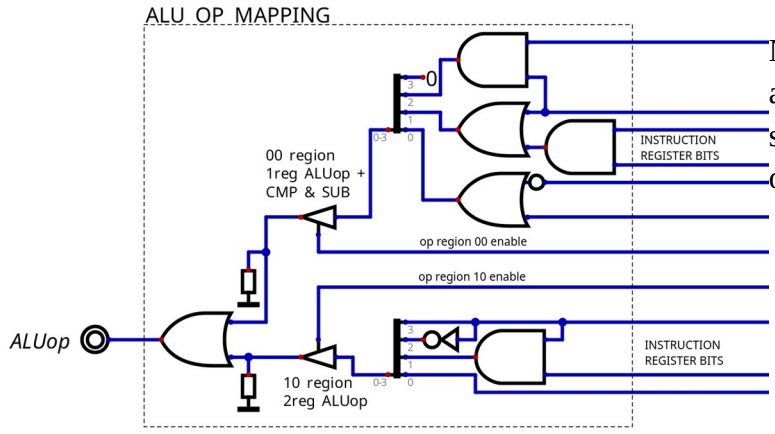



*Figure 4.8 ALU-op mapping implementation in Digital.*

## 4.4 Register File

Due to the ALP8B only having four general purpose registers, with a single special register for the condition flags. The bottom 4 bits in the IR are used to decode RA/RB, with additional AND gating from the microcode for activation. The flags register is enabled directly through microcode (as seen in Figure 4.3)

RA Output enable (RAOE) is gated with AND NOT PCWE. This is because a single PC increment - a microcode with a destination, but no source due to the internal adder used for the PC – defaults to using RA as source (as RA source is ‘00’). Thus, a simple AND gate is used to ensure that a microcode sequence to increase the program counter by one does not unintentionally enable the output of RA. Ultimately, this is the consequence of poor multi-word instruction planning.

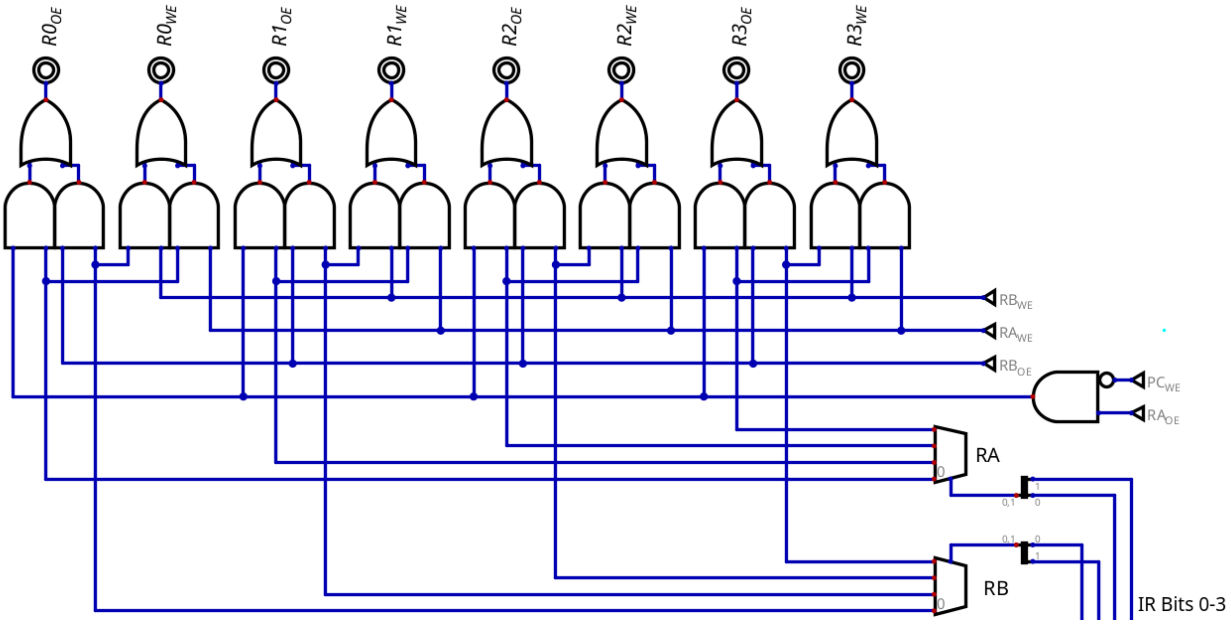



*Figure 4.9: GPR selection inside the Control Unit*

### 4.4.1 Flags Register

The Flags register is designed as a separate two flip flop register. The register has two inputs, Write Enable Z (WE_Z), which functions as a typical write enable, and Write Enable C (WE_C). WE_C is special in that it is a two bit write enable which drives a multiplexer to select a given C value.

| WE_C Bits [1:0] | Outcome |
|---|---|
| `00` | Preserve previous value |
| `01` | Overwrite current with ALU output |
| `10` | Overwrite current with 0 |
| `11` | Overwrite current with 1 |

The implementation of the flags register in digital can be seen below. A reset component (the box with the R at the bottom) is used to initialise the flip flop to 0 at simulation start, such that the user does not have to waste a byte of memory clearing the flags on program start

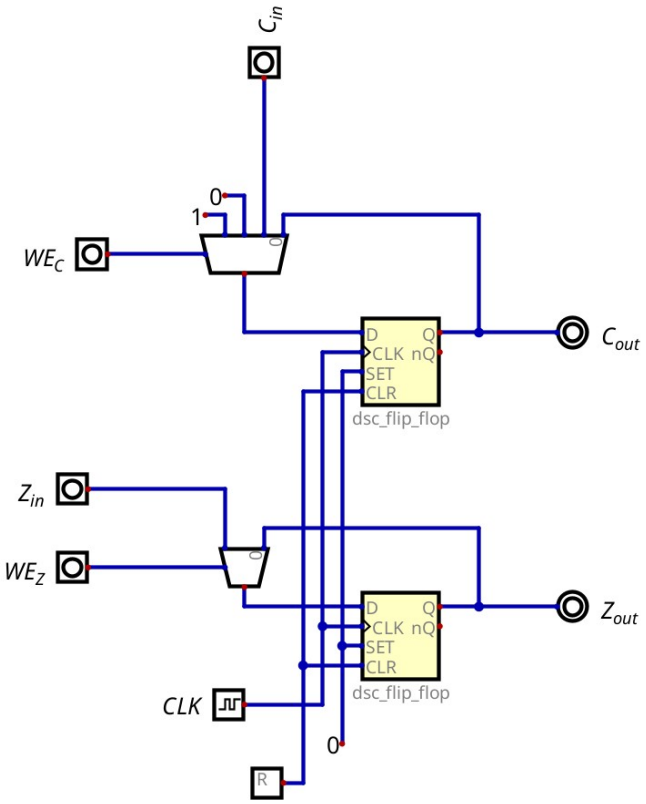



*Figure 4.10: Flags Register*

# 5 Software

## 5.1 Microcode Programming

The microcode sequences were programmed via a python script to simplify the process. The finalised output is an Intel HEX file that is read by Digital as the microcode ROM's data. A snippet can be seen below.

```
rom = [0] * 64

def ucode(opcode, t_state, control_word):
      # address = (OPC shifted left 2 bits) OR (T-state)
      address = (opcode << 2) | t_state

      # sore the word in the final rom , make sure it has the validity bit
      rom[address] = control_word | VALID_INSTR

# OPC_LOAD
ucode(OPC_LDM, T2, DST_MAR | SC_RB) # load address value from B into MAR
ucode(OPC_LDM, T3, ADDR_SEL_CFLGS | DST_RA | SC_RAM) # use MAR address to
load RAM value into RA
ucode(OPC_LDM, T4, T_RESET)

# ... other sequences
```

## 5.2 Assembler

A bare-bones, two pass assembler was also created in python to compile the assembly to machine code. The assembler has support for:

- Labels, for assembly code organisation.
- Comments.
- An '`INI`' pseudo-instruction to initialise any memory location to a given initial value.

The assembler outputs to Intel HEX (for the Digital Simulator), and also to a line by line hex / binary debug file.

## 5.3 Demoscene Programs

A variety of demoscene programs can be found in the root of the project repository under '`programs/`'. See the README for short gif overviews on what each program does.

# 6 Learnings & Next steps

Similar to what was mentioned previously in the ISA section, much of this project was not designed with future expandability in mind. There were also a plethora of poor design choices, both in the hardware (simulation) and in the ISA that had cascading consequences throughout the project. A majority of this can be attributed to naivety, however, had greater care been taken in the planning phase, these poor choices could've been minimised. Much of this project was also based off problem solving these poor choices - and hence creating new problems along the way. Utilising a greater amount of computer architecture theory would've helped supplement good design decisions more often.

Ideally, future development would consist of a full ground up redesign, taking into account the learnings from this project. Version 2 would ideally consist of the following changes/additions:

- Larger memory space (16-bit addressed).
- Hardware stack/base pointer.
- Memory mapped I/O.
- Greater range of instructions: Immediate values, Math without carry, signed/unsigned flags.
- Complex branching: program counter relative jumps!, RISC-style comparison within the branch instruction.
- Larger register file.
- Greater adherence to orthogonality within the ISA.
- Increased variety of testing (development via Verilog / C++)